

4. Complete Test Plan

Group 1: Register & BD Access Tests

Test	Description	RAL Method
tc_reg_reset_values	Verify all 21 registers match spec reset values after RST_I	uvm_reg_hw_reset_seq
tc_reg_walking_one	Walking-1 across all RW bits, single WISHBONE cycles	uvm_reg_bit_bash_seq
tc_reg_walking_zero	Walking-0 across all RW bits	uvm_reg_bit_bash_seq
tc_reg_max_values	Write max value, read back, hard reset, verify	Custom seq
tc_reg_read_only_fields	Verify R-only fields (MIIRX_DATA, MIISTATUS bits) reject writes	Custom seq
tc_bd_walking_one	Walking-1 across BD RAM (0x400–0x7FF)	uvm_mem_walk_seq
tc_bd_reset_preservation	BD RAM holds values after soft reset (RST bit), clears after hard reset	Custom seq
tc_bd_boundary	Access first BD (0x400), last BD (0x7FC), and out-of-range	uvm_mem_access_seq
tc_tx_bd_num_split	Write TX_BD_NUM=0x32 → verify 50 TX BDs and 78 RX BDs are accessible	Custom seq
tc_bd_access_during_reset	Attempt BD access while RST bit set → expect no access	Custom seq

Group 2: MIIM (MII Management) Tests

Test	Description
tc_miim_clkdiv_all	Test all 255 clock divider values in MIIMODER.CLKDIV; verify MDC frequency = $\text{CLK}/(2 \times \text{CLKDIV})$
tc_miim_write_phy_reg	Write to PHY control register (addr 0x0) via MIICOMMAND.WCTRLDATA; verify MDIO frame format
tc_miim_read_phy_reg	Read from PHY status register (addr 0x1); verify PRSD in MIIRX_DATA
tc_miim_read_nonwritable	Attempt write to read-only PHY register; verify PHY ignores it
tc_miim_reset_phy	Assert PHY reset through MIIM write; verify PHY responds
tc_miim_walk_phy_addr	Walking-1 across FIAD[4:0] with and without

	preamble
tc_miim_walk_reg_addr	Walking-1 across RGAD[4:0] with and without preamble
tc_miim_walk_data	Walking-1 across 16-bit write data with and without preamble
tc_miim_wrong_phy_addr	Read from non-existent PHY address → MDIO high-Z, verify MIIRX_DATA undefined
tc_miim_wrong_write_correct_read	Write to wrong PHY addr, read from correct addr; verify no corruption
tc_miim_busy_nvalid_write	Verify BUSY asserts during write, deasserts at end; NVALID behavior
tc_miim_busy_nvalid_read	Verify BUSY/NVALID durations during read operation
tc_miim_busy_nvalid_scan	Verify BUSY stays asserted during continuous scan; NVALID clears after first scan
tc_miim_scan_linkfail	Scan PHY status; detect LINKFAIL asserted/deasserted; verify MIISTATUS.LINKFAIL
tc_miim_scan_linkfail_sliding	Sliding LINKFAIL bit during scan — verify edge detection
tc_miim_stop_scan_after_read	Assert SCANSTAT then immediately clear; sliding stop with/without preamble
tc_miim_stop_scan_after_write	Stop scan immediately after write request; sliding
tc_miim_stop_scan_after_scan	Stop scan after 1st and 2nd scan iterations; sliding
tc_miim_no_preamble	Set MIIMODER.MIINOPRE=1; verify 32-bit preamble absent on MDIO

Group 3: Frame Transmission Tests

Test	Description
tc_tx_basic_frame	Transmit single minimum-length frame (64B); verify preamble + SFD + data + CRC on MII
tc_tx_max_frame	Transmit maximum-length frame (1518B); verify no truncation
tc_tx_short_frame_pad_on	Frame < MINFL with PAD=1; verify padded to 64B with zeros + CRC
tc_tx_short_frame_pad_off	Frame < MINFL with PAD=0; verify transmitted as-is (no padding)
tc_tx_crc_enable	CRCEN=1; verify 4-byte CRC appended to frame
tc_tx_crc_disable	CRCEN=0; verify no CRC appended
tc_tx_delayed_crc	DLYCRCEN=1; CRC calculation starts 4 bytes after SFD

tc_tx_huge_frame	HUGEN=1; transmit frame > 1518B; verify transmission completes
tc_tx_huge_disabled	HUGEN=0; transmit frame > MAXFL; verify abort at max
tc_tx_no_preamble	NOPRE=1 in MODER; verify 7-byte preamble suppressed
tc_tx_ipg_back_to_back	Transmit back-to-back frames; verify IPG $\geq 0.96\mu\text{s}$ (IPGT register value)
tc_tx_ipg_non_back_to_back	Verify IPGR1 and IPGR2 windows respected
tc_tx_multi_bd_chain	Chain 8 TX BDs (WR=0 until last); verify continuous transmission
tc_tx_wrap_bit	Last BD has WR=1; verify circular wrap to first BD
tc_tx_underrun	Throttle WISHBONE master to starve TX FIFO; verify TxAbort + UR bit in TX BD
tc_tx_done_interrupt	IRQ=1 in TX BD; verify TXB interrupt in INT_SOURCE after transmission
tc_tx_error_interrupt	Force underrun with IRQ=1; verify TXE interrupt
tc_tx_retry_count	In half-duplex: inject collision; verify RTRY field incremented in TX BD
tc_tx_max_retry	Inject MAX_RET+1 collisions; verify RL bit set, transmission aborted
tc_tx_late_collision	Inject collision after COLLVALID window; verify LC bit + abort
tc_tx_excessive_deferral	EXDFREN=0; keep carrier busy > defer limit; verify abort
tc_tx_excessive_deferral_wait	EXDFREN=1; verify TX waits indefinitely for carrier
tc_tx_loopback	LOOPBCK=1; verify transmitted data appears on RX path
tc_tx_carrier_sense_lost	Assert MCrS then deassert mid-frame; verify CS bit in TX BD
tc_tx_only_4byte_min	TX BD with LEN ≤ 4 ; verify frame not transmitted (spec note)
tc_tx_unaligned_ptr	TX buffer pointer at non-word-aligned address; verify correct byte extraction

Group 4: Frame Reception Tests

Test	Description
tc_rx_basic_frame	PHY sends standard frame; verify assembly and storage in memory via DMA
tc_rx_crc_check_pass	Valid CRC frame; verify no CRC error in RX BD
tc_rx_crc_check_fail	Corrupt last byte of CRC; verify CRC bit set in RX BD
tc_rx_short_frame_reject	Frame < MINFL; RECSMALL=0; verify frame dropped
tc_rx_short_frame_accept	Frame < MINFL; RECSMALL=1; verify SF bit set in RX

	BD
tc_rx_too_long_frame	Frame > MAXFL; HUGEN=0; verify reception stops at MAXFL; TL bit set
tc_rx_huge_frame_enable	Frame > MAXFL; HUGEN=1; verify full frame received
tc_rx_preamble_removal	Verify 7-byte preamble + SFD stripped; only payload in memory
tc_rx_short_preamble	PHY sends frame with <7-byte preamble (only SFD); verify accepted
tc_rx_delayed_crc	DLYCRCEN=1; verify CRC checking starts 4 bytes after SFD
tc_rx_dribble_nibble	PHY sends odd-nibble-count frame; verify DN and CRC bits in RX BD
tc_rx_invalid_symbol	PHY sets MRxD=0xE (100Mbps error code); verify IS bit in RX BD
tc_rx_phy_error	MRxErr asserted mid-frame; verify frame aborted, no error in BD
tc_rx_overrun	Throttle WISHBONE master; RX FIFO overrun; verify OR bit in RX BD
tc_rx_ifg_minimum	Two back-to-back frames with IFG < 960ns; verify second frame dropped
tc_rx_ifg_disable	r_IFG=1 in MODER; verify frames accepted regardless of IFG
tc_rx_multi_bd	Receive frames filling multiple RX BDs; verify wrap using WR bit
tc_rx_no_empty_bd	All RX BDs marked non-empty; verify frame discarded, BUSY interrupt
tc_rx_interrupt_rxb	IRQ=1 in RX BD; verify RXB interrupt after frame received
tc_rx_interrupt_rxe	Receive with error, IRQ=1; verify RXE interrupt
tc_rx_late_collision	Late collision during reception; verify LC bit in RX BD
tc_rx_unaligned_ptr	RX buffer pointer at non-word-aligned address; verify correct byte select
tc_rx_only_4byte_min	Frame ≤ 4 bytes; verify received with CRC error (per spec note)

Group 5: Address Recognition Tests

Test	Description
tc_addr_unicast_match	Frame dest = MAC_ADDR0/1; verify accepted
tc_addr_unicast_mismatch	Frame dest ≠ MAC address; PRO=0; verify dropped (RxAbort)
tc_addr_promiscuous	PRO=1; send frame with foreign dest; verify accepted + M bit set in RX BD
tc_addr_broadcast_accept	BRO=0; send broadcast (FF:FF:FF:FF:FF:FF); verify

	accepted
tc_addr_broadcast_reject	BRO=1; send broadcast; PRO=0; verify rejected
tc_addr_multicast_hash	IAM=1; load HASH0/HASH1; send multicast matching hash; verify accepted
tc_addr_multicast_hash_miss	IAM=1; multicast not in hash table; verify dropped
tc_addr_miss_bit	PRO=1; frame doesn't match MAC; verify M=1 in RX BD

Group 6: Flow Control Tests (Full Duplex)

Test	Description
tc_fc_rx_pause_frame	PHY sends valid PAUSE frame (type=8808, op=0001); RXFLOW=1; verify TX paused for timer value
tc_fc_rx_pause_timer_expire	PAUSE timer counts down to 0; verify TX resumes
tc_fc_rx_pause_passall_0	PASSALL=0, RXFLOW=1; PAUSE frame not stored to memory; RXC interrupt set
tc_fc_rx_pause_passall_1	PASSALL=1, RXFLOW=1; PAUSE frame stored to memory + CF bit set + RXC interrupt
tc_fc_rx_pause_passall_1_rxflow_0	PASSALL=1, RXFLOW=0; stored as normal frame, RXB set, no pause
tc_fc_rx_pause_both_0	PASSALL=0, RXFLOW=0; PAUSE frame silently dropped
tc_fc_tx_pause_request	Write TXPAUSERQ=1 to TXCTRL with TXPAUSETV; verify PAUSE frame transmitted with correct timer value
tc_fc_tx_pause_frame_format	Verify PAUSE frame structure: dest=01-80-c2-00-00-01, type=8808, op=0001, 42B reserved, CRC
tc_fc_tx_pause_blocked	TXFLOW=0; write TXPAUSERQ=1; verify no PAUSE frame sent
tc_fc_rx_invalid_pause	Receive frame with wrong type/opcode; verify not treated as PAUSE
tc_fc_slot_timer	Verify slot timer generates correct pulse intervals (64-byte slot time at 10/100 Mbps)
tc_fc_txc_interrupt	TXFLOW=1; send PAUSE frame; verify TXC interrupt in INT_SOURCE
tc_fc_rxc_interrupt	RXFLOW=1; receive PAUSE frame; verify RXC interrupt in INT_SOURCE

Group 7: Half-Duplex / CSMA-CD Tests

Test	Description
tc_hd_collision_backoff	FULLD=0; inject collision; verify jam pattern (0x99999999) + binary exponential backoff
tc_hd_collision_retry	Inject up to 15 collisions; verify each retry uses wider backoff window; TxRetry signal
tc_hd_collision_window	Verify ColWindow = COLLVALID bytes from preamble
tc_hd_nobckof	NOBCKOF=1; verify immediate retransmission after collision (no backoff)
tc_hd_carrier_sense_defer	MCrS asserted; verify TX defers; DF bit set after eventual success
tc_hd_carrier_sense_ifg	Carrier appears within IPGR1 window; verify counter reset

Group 8: Interrupt System Tests

Test	Description
tc_int_mask_all	INT_MASK=0x00; trigger all events; verify INTA_O never asserts
tc_int_unmask_txb	INT_MASK.TXB_M=1; transmit frame; verify INTA_O asserts
tc_int_unmask_rxb	INT_MASK.RXF_M=1; receive frame; verify INTA_O asserts
tc_int_clear_by_write1	Write 1 to INT_SOURCE bits to clear; verify bits clear and INTA_O deasserts
tc_int_busy	Fill all RX BDs; receive new frame; verify BUSY bit in INT_SOURCE
tc_int_multiple_sources	Trigger TX + RX simultaneously; verify all bits set, single INTA_O

Group 9: Reset & Clock Domain Tests

Test	Description
tc_rst_hard_reset	Assert RST_I; verify all registers return to reset values; TX/RX halt
tc_rst_mid_tx	Assert RST_I during frame transmission; verify graceful abort
tc_rst_mid_rx	Assert RST_I during frame reception; verify BD not corrupted
tc_rst_bd_preservation	BD RAM holds values after RST_I (per design doc section 3.4.1)
tc_clk_10mbps	Set PHY clocks to 2.5MHz (10Mbps); full TX/RX test
tc_clk_100mbps	Set PHY clocks to 25MHz (100Mbps); full TX/RX test
tc_clk_no_link	MRxClk random 2-40MHz (no link); verify MAC does not lock up

Group 10: Stress & Corner Case Tests

Test	Description
tc_stress_all_bd_tx	Fill all 128 TDs as TX BDs (TX_BD_NUM=0x80); transmit burst
tc_stress_all_bd_rx	TX_BD_NUM=0x00; all BDs as RX; receive burst
tc_stress_simultaneous_txrx	Full duplex: simultaneous TX burst and RX burst
tc_stress_wb_wait_states	Add random wait states to WISHBONE slave; verify no deadlock
tc_stress_bd_boundary_addr	Access TX BDs at 0x400, RX BDs ending at 0x7FC; boundary check
tc_stress_random_frame_size	Randomize frame sizes from 5 to 1600 bytes; run 1000 frames
tc_stress_fifo_depth	Fill TX FIFO to capacity (16 words) then drain; verify no stall

1- tc_reg_reset_values

After asserting RST_I, every one of the 21 configuration registers must read back its documented reset value as per the specification. Uses the UVM RAL built-in reset sequence so no manual register reads are needed.

COMPONENTS USED

`wb_slave_agent` `eth_reg_block (RAL)` `uvm_reg_hw_reset_seq` `eth_coverage`

TEST FLOW

- 1-TestAssert RST_I=1 for 5 clock cycles via `wb_slave_agent`
- 2-TestDeassert RST_I=0, wait 2 clock cycles
- 3-`uvm_reg_hw_reset_seq` iterates every register in `eth_reg_block`; calls `reg.read()` via frontdoor WISHBONE
- 4-`wb_slave_driver` Drives ADDR_I, WE_I=0, STB_I, CYC_I; waits for ACK_O
- 5-RAL predictor Compares read value against `reg.get_reset()` for every field
- 6-`eth_coverage`Samples `reg_reset_cg` covergroup — all register names

PASS CRITERIA

MODER reset = 0x0000_A000 (NOPRE, NOBCKOF, FULLD bits check)
IPGT reset = 0x12, IPGR1 = 0x0C, IPGR2 = 0x12
PACKETLEN reset = 0x0040_0600
COLLCONF reset = 0x000F_003F
TX_BD_NUM reset = 0x40
MIIMODER reset = 0x64
All status/interrupt regs (INT_SOURCE, MIISTATUS) reset = 0x0
`uvm_reg_hw_reset_seq` reports zero mismatches

uvm_reg_hw_reset_seq

It is a **pre-built UVM sequence**

Its entire job is one thing:

Read every register in your **RAL model via the actual bus (frontdoor)**, compare each value against the reset value your RAL model declares, and report any mismatch as a UVM error.

The Actual Source Code (Simplified)

Here is what the sequence actually does internally, stripped to its essence:

systemverilog

```
class uvm_reg_hw_reset_seq extends uvm_reg_sequence;

    uvm_reg_block model; // you set this — points to your eth_reg_block

    task body();

        uvm_reg regs[$]; // queue to hold all register handles
        uvm_reg_data_t rdata; // value read from hardware
        uvm_reg_data_t exp; // expected reset value from RAL model
        uvm_status_e status;

        // Step 1: collect every register from the block
        model.get_registers(regs);

        // Step 2: for each register
        foreach (regs[i]) begin

            // Step 3: skip registers with no reset value defined
            if (!regs[i].has_reset()) continue;

            // Step 4: read the register from actual hardware
            // this goes through your adapter → WISHBONE bus
            regs[i].read(status, rdata, UVM_FRONTDOOR);

            // Step 5: get what the reset value SHOULD be
            exp = regs[i].get_reset();

            // Step 6: compare — only check bits that have reset values
            // (reserved bits are masked out automatically)
            if ((rdata & regs[i].get_reset("HARD"))
                != regs[i].get_reset("HARD")) begin

                `uvm_error("REG_HW_RESET",
                    $sformatf("Register '%s' expected 0x%0h after reset, got 0x%0h",
```



```

        regs[i].get_full_name(), exp, rdata))
    end
end

endtask

endclass

```

For MODER (offset 0x00): CYC_I = 1 STB_I = 1 WE_I = 0 ← read SEL_I = 4'hF ADDR_I = 0x00

DUT responds: ACK_O=1, DATA_O = current register value

adapter captures DATA_O, returns to RAL

RAL compares against 0x0000_A000

2-tc_reg_walking_one

Shifts a single 1-bit through every RW-accessible bit position in every register. For each bit position: write 1 to that bit only, read back, verify; then restore to reset value. Uses `uvm_reg_bit_bash_seq`.

COMPONENTS USED

<code>wb_slave_agent</code>	<code>eth_reg_block (RAL)</code>	<code>uvm_reg_bit_bash_seq</code>
-----------------------------	----------------------------------	-----------------------------------

TEST FLOW

1 `uvm_reg_bit_bash_seq` For each RW register: writes all zeros (frontdoor), reads back, checks

2 `uvm_reg_bit_bash_seq` For each RW bit position N: writes 1 << N, reads back

3 `wb_slave_driver` Each write: drives ADDR_I=reg_offset, DATA_I=value, WE_I=1, STB_I, CYC_I; waits ACK_O

4 RAL predictor Verifies read data matches written value; flags W1C or read-only bits that differ

5 `uvm_reg_bit_bash_seq` Writes all ones, reads back; checks RO bits unchanged, RW bits set

PASS CRITERIA

Every RW bit in all 21 registers accepts 0→1 and 1→0 transitions

Read-only bits (MIIRX_DATA[15:0], MIISTATUS[2:0]) never change on write

No ERR_O asserted for valid register addresses with SEL_I=4'hF

ERR_O asserted when SEL_I ≠ 4'hF on any register access

What `uvm_reg_bit_bash_seq` Actually Does Internally

The UVM library sequence does more than just walking 1. It runs four sub-patterns on every register. Here is its actual internal logic:

```
systemverilog
```

```
// Inside uvm_reg_bit_bash_seq (UVM library)
```

```
task body();
```

```
    uvm_reg regs[$];  
    model.get_registers(regs); // collect all registers
```

```
    foreach (regs[i]) begin  
        bash_kth_reg(regs[i]); // test this register completely  
    end
```

```
endtask
```

```
task bash_kth_reg(uvm_reg rg);
```

```
    int n_bits;  
    uvm_reg_field fields[$];  
    uvm_reg_data_t dc_mask; // don't-care mask (reserved + RO bits)  
    uvm_reg_data_t reset_val;
```

```
    n_bits = rg.get_n_bytes() * 8; // 32 for your registers  
    reset_val = rg.get_reset();  
    rg.get_fields(fields);
```

```
    // Build a mask of bits we should NOT touch or check  
    // (reserved bits, read-only bits, write-only bits)  
    dc_mask = 0;  
    foreach (fields[i]) begin  
        if (fields[i].get_access() != "RW")  
            dc_mask |= fields[i].get_mask();  
        //      ^ set those bit positions in the don't-care mask  
    end
```

```
    // — Pattern 1: Walking 1 —————  
    for (int k = 0; k < n_bits; k++) begin
```

```
        if (dc_mask[k]) continue; // skip reserved/RO bits
```

```
        bash_bit(rg, k, 1, reset_val, dc_mask);  
        //      ↑ ↑  
        //      bit value to write  
    end
```

```
    // — Pattern 2: Walking 0 —————  
    // First write all 1s to every RW bit  
    rg.write(status, ~dc_mask);
```

```

for (int k = 0; k < n_bits; k++) begin

    if (dc_mask[k]) continue;

    bash_bit(rg, k, 0, reset_val, dc_mask);
    //      ↑ ↑
    //      bit value to write (0 this time)
end

endtask

task bash_bit(uvm_reg  rg,
              int      k,      // which bit position
              bit      val,    // 1 for walking-1, 0 for walking-0
              uvm_reg_data_t reset_v,
              uvm_reg_data_t dc_mask);

    uvm_reg_data_t wr_val;
    uvm_reg_data_t rd_val;
    uvm_reg_data_t exp_val;
    uvm_status_e   status;

    // — Step A: Build the write value —
    if (val == 1)
        wr_val = (1 << k);      // only bit k is 1, all others 0
    else
        wr_val = ~(1 << k) & ~dc_mask; // only bit k is 0, all RW bits 1

    // — Step B: Write to hardware (frontdoor WISHBONE) —
    rg.write(status, wr_val);

    // — Step C: Read back from hardware (frontdoor WISHBONE) —
    rg.read(status, rd_val);

    // — Step D: Build expected value —
    // For RW bits: expect exactly what you wrote
    // For RO/reserved bits: expect their current value (unchanged)
    exp_val = (wr_val & ~dc_mask) | (reset_v & dc_mask);

    // — Step E: Compare —
    if ((rd_val & ~dc_mask) != (exp_val & ~dc_mask)) begin
        `uvm_error("REG_BIT_BASH",
            $sformatf("Reg '%s' bit %0d: wrote 0x%0h, expected 0x%0h, got 0x%0h",
                rg.get_full_name(), k, wr_val, exp_val, rd_val))
    end

    // — Step F: Restore register to reset value —
    rg.write(status, reset_v);

endtask

```

Walking Through a Concrete Example — MODER Register

MODER is 32 bits wide. Reset value = 0x0000_A000.

Let us map out which bits are RW vs reserved:

Bit 31–17: Reserved → dc_mask bits 31–17 = 1 (skip these)
Bit 16: RECSMALL → RW → test this
Bit 15: PAD → RW → test this
Bit 14: HUGEN → RW → test this
Bit 13: CRCEN → RW → test this (reset=1)
Bit 12: DLYCRCEN → RW → test this
Bit 11: Reserved → dc_mask bit 11 = 1 (skip)
Bit 10: FULLD → RW → test this
Bit 9: EXDFREN → RW → test this
Bit 8: NOBCKOF → RW → test this (reset=1)
Bit 7: LOOPBCK → RW → test this
Bit 6: IFG → RW → test this
Bit 5: PRO → RW → test this
Bit 4: IAM → RW → test this
Bit 3: BRO → RW → test this
Bit 2: NOPRE → RW → test this
Bit 1: TXEN → RW → test this
Bit 0: RXEN → RW → test this

dc_mask = 0xFFFF_0800 (bits 31–17 and bit 11)

Now the sequence walks through 32 bit positions for MODER. Let us trace every interesting one:

Bit 0 — RXEN

k = 0

Step A: wr_val = (1 << 0) = 0x0000_0001

Step B: WISHBONE write

ADDR_I = 0x00 (MODER offset)

DATA_I = 0x0000_0001

WE_I = 1

SEL_I = 0xF

STB_I = 1

CYC_I = 1

→ wait for ACK_O

Step C: WISHBONE read

ADDR_I = 0x00

WE_I = 0

→ DUT returns DATA_O

Step D: exp_val = (0x0000_0001 & ~0xFFFF0800)
| (0x0000A000 & 0xFFFF0800)

```
= 0x0000_0001 & 0x0000_F7FF
| 0x0000_A000 & 0xFFFF_0800
= 0x0000_0001
| 0x0000_A000 ← bits 13 and 8 are reset=1, in dc_mask? NO
    wait — bits 13,8 are RW, not reserved
    dc_mask only covers reserved bits
= 0x0000_0001
(reserved bits stay 0, RW bits we only wrote RXEN=1)
```

Step E: DUT read returns 0x0000_0001 → PASS ✓

Step F: restore: write 0x0000_A000 (MODER reset value)

Wait — step D needs clarification. The expected value for RW bits is exactly what you wrote. You wrote 0x0000_0001. Only RXEN=1. All other RW bits including CRCEN and NOBCKOF are 0. So you expect to read back 0x0000_0001. The restore in Step F puts MODER back to 0x0000_A000 before the next iteration.

Bit 11 — Reserved

k = 11

dc_mask[11] = 1 → continue (skip this bit entirely)

No write. No read. No compare.
The sequence just moves to k=12.

Bit 13 — CRCEN (reset = 1)

k = 13

Step A: wr_val = (1 << 13) = 0x0000_2000

Step B: WISHBONE write
ADDR_I = 0x00
DATA_I = 0x0000_2000 ← only CRCEN=1, everything else 0
WE_I = 1

Step C: WISHBONE read
DUT returns DATA_O = 0x0000_2000

Step D: exp_val = 0x0000_2000 (just CRCEN bit set)

Step E: 0x0000_2000 == 0x0000_2000 → PASS ✓

Step F: restore: write 0x0000_A000

The Walking-0 Pattern (Part 2 of the Same Sequence)

After all 32 walking-1 iterations for MODER, the sequence runs walking-0. First it writes all 1s to every RW bit:

Write all-RW-ones:

DATA_I = ~dc_mask = ~0xFFFF_0800 = 0x0000_F7FF

MODER now = 0x0000_F7FF

(every RW bit is 1: RECSMALL, PAD, HUGEN, CRCEN, DLYCRCEN, FULLD, EXDFREN, NOBCKOF, LOOPBCK, IFG, PRO, IAM, BRO, NOPRE, TXEN, RXEN all = 1)

Then for each RW bit position, it clears that one bit to 0:

Walking-0 for bit 0 (RXEN):

k = 0, val = 0

Step A: wr_val = ~(1 << 0) & ~dc_mask
= 0xFFFF_FFFE & 0x0000_F7FF
= 0x0000_F7FE
(every RW bit = 1 EXCEPT RXEN which is 0)

Step B: write 0x0000_F7FE to MODER

Step C: read back from DUT

Step D: exp_val = 0x0000_F7FE

Step E: compare → PASS ✓

Step F: restore to 0x0000_A000

Now Let Us See Every WISHBONE Transaction For Just IPGT

IPGT is simpler — only 7 bits, all RW, reset = 0x12 = 0001_0010.

dc_mask = 0xFFFF_FF80 (bits 31–7 are reserved or don't exist)

Walking-1 iterations: 7 total (bits 0–6)

Walking-0 iterations: 7 total (bits 0–6)

Total WISHBONE cycles: 7×(write+read+restore) + setup + 7×(write+read+restore)
= 7×3 + 1 + 7×3 = 43 WISHBONE cycles just for IPGT

Here is every transaction:

WALKING-1 FOR IPGT:

—— Bit 0 ———

Write: ADDR=0x0C DATA=0x00000001 WE=1 SEL=0xF → ACK

Read: ADDR=0x0C WE=0 → DATA=0x00000001 ACK

compare 0x01 vs 0x01 → PASS

Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 1 ——

Write: ADDR=0x0C DATA=0x00000002 WE=1 SEL=0xF → ACK
 Read: ADDR=0x0C WE=0 → DATA=0x00000002 ACK
 compare 0x02 vs 0x02 → PASS
 Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 2 ——

Write: ADDR=0x0C DATA=0x00000004 WE=1 SEL=0xF → ACK
 Read: ADDR=0x0C WE=0 → DATA=0x00000004 ACK
 compare 0x04 vs 0x04 → PASS
 Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 3 ——

Write: ADDR=0x0C DATA=0x00000008 WE=1 SEL=0xF → ACK
 Read: ADDR=0x0C WE=0 → DATA=0x00000008 ACK
 compare 0x08 vs 0x08 → PASS
 Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 4 ——

Write: ADDR=0x0C DATA=0x00000010 WE=1 SEL=0xF → ACK
 Read: ADDR=0x0C WE=0 → DATA=0x00000010 ACK
 compare 0x10 vs 0x10 → PASS
 Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 5 ——

Write: ADDR=0x0C DATA=0x00000020 WE=1 SEL=0xF → ACK
 Read: ADDR=0x0C WE=0 → DATA=0x00000020 ACK
 compare 0x20 vs 0x20 → PASS
 Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 6 ——

Write: ADDR=0x0C DATA=0x00000040 WE=1 SEL=0xF → ACK
 Read: ADDR=0x0C WE=0 → DATA=0x00000040 ACK
 compare 0x40 vs 0x40 → PASS
 Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

Bits 7–31: all in dc_mask → skipped entirely

WALKING-0 SETUP FOR IPGT:

Write: ADDR=0x0C DATA=0x0000007F WE=1 SEL=0xF → ACK
 (all 7 RW bits = 1: 0111_1111)

—— Bit 0 (RXEN walking-0) ——

Write: ADDR=0x0C DATA=0x0000007E WE=1 SEL=0xF → ACK
 (0111_1111 with bit 0 cleared = 0111_1110)
 Read: ADDR=0x0C WE=0 → DATA=0x0000007E ACK
 compare 0x7E vs 0x7E → PASS
 Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

... (repeat for bits 1–6)

What Bugs This Test Catches

Each failure points to a very specific RTL problem:

Failure type	What it means in RTL
Read back all zeros always	Register not connected to DATA_O path Wrong address decode in eth_registers.v
Read back all ones always	DATA_O stuck high Missing output enable
Bit N always reads 0	That specific flip-flop not written Bit N of DATA_I not connected to FF input
Bit N always reads 1	FF reset value wrong, or bit stuck
Walking-1 PASS but walking-0 FAIL on same bit	Register has bit coupling — writing one bit affects another (logic bug in DUT)
Reserved bits change when RW bits written	Reserved bit positions not properly masked in eth_registers.v
Value does not restore after Step F	Restore write not working (same as write failure but only on the reset value pattern)
ERR_O asserts on valid register address	SEL_I checking too strict, or address decode bug in eth_wishbone.v

Total Bus Transactions Across All 21 Registers

Per register:

Walking-1: (number of RW bits) $\times$ 3 transactions (write + read + restore)

Walking-0: 1 setup write + (number of RW bits) $\times$ 3 transactions

For your 21 registers (approximate RW bit counts):

MODER 15 RW bits $\rightarrow 15 \times 3 + 1 + 15 \times 3 = 91$

INT_SOURCE 7 RW bits $\rightarrow 7 \times 3 + 1 + 7 \times 3 = 43$

INT_MASK 7 RW bits $\rightarrow 43$

IPGT 7 RW bits $\rightarrow 43$

IPGR1 7 RW bits $\rightarrow 43$

IPGR2 7 RW bits $\rightarrow 43$

PACKETLEN 32 RW bits $\rightarrow 32 \times 3 + 1 + 32 \times 3 = 193$

COLLCONF 14 RW bits $\rightarrow 85$

TX_BD_NUM 8 RW bits $\rightarrow 49$

CTRLMODER 3 RW bits $\rightarrow 19$

MIIMODER 9 RW bits $\rightarrow 55$

MIICOMMAND 3 RW bits $\rightarrow 19$

MIIADDRESS 10 RW bits $\rightarrow 61$

MIITX_DATA 16 RW bits $\rightarrow 97$

MAC_ADDR0 32 RW bits $\rightarrow 193$

MAC_ADDR1 16 RW bits $\rightarrow 97$

HASH0 32 RW bits $\rightarrow 193$

HASH1 32 RW bits $\rightarrow 193$

TXCTRL 17 RW bits $\rightarrow 103$

Total $\approx$ 1,663 WISHBONE transactions

At $\sim$ 3 clock cycles each with no wait states $\approx$ 5,000 clock cycles

At 100MHz system clock = 50 microseconds of simulation time

This is why `uvm_reg_bit_bash_seq` is always run early in regression — it is fast, completely automatic, and catches a huge class of RTL connectivity bugs before you even attempt functional tests.

3- tc_bd_walking_one

Exercises all 1024 bytes of the internal BD RAM using `uvm_mem_walk_seq`. Verifies that the RAM correctly stores and returns data at every address without corruption. BD RAM is only accessible when the MAC is not in reset.

COMPONENTS USED

```
wb_slave_agent eth_bd_mem (uvm_mem) uvm_mem_walk_seq
```

TEST FLOW

1TestEnsure MAC is not in reset (RST bit in MODER cleared)

2uvm_mem_walk_seqWrites walking-1 pattern to every 32-bit word from offset 0x400 to 0x7FC

3wb_slave_driverEach write: ADDR_I = 0x400+offset, DATA_I = pattern, WE_I=1, SEL_I=0xF

4uvm_mem_walk_seqReads back every location and compares

5TestAssert RST_I, then read BD RAM — verify values preserved

PASS CRITERIA

All 256 32-bit words (0x400–0x7FC) store and return correct values

No ACK timeout for any BD address within 0x400–0x7FF range

Addresses outside 0x400–0x7FF do not alias into BD RAM

RAM values preserved across soft reset (RST_I) as per spec

READY bit (RD/E) prevents host access once set — verify ERR_O or no effect when written

4- tc_tx_bd_num_split

Writes 0x32 (50) to TX_BD_NUM register and verifies that exactly 50 TX BDs are accessible at 0x400–0x58C and 78 RX BDs are accessible at 0x590–0x7FC. Tests the boundary address for each region.

COMPONENTS USED

wb_slave_agent	eth_reg_block (RAL)	eth_bd_mem (uvm_mem)	eth_tx_bd_num_seq
----------------	---------------------	----------------------	-------------------

TEST FLOW

- 1eth_tx_bd_num_seqWrite TX_BD_NUM = 0x32 via RAL frontdoor
- 2eth_tx_bd_num_seqWrite all 50 TX BDs: address range 0x400 to 0x58C (8 bytes each × 50 = 0x190)
- 3eth_tx_bd_num_seqWrite all 78 RX BDs: address range 0x590 to 0x7FC
- 4eth_tx_bd_num_seqRead back all 128 BDs and verify no overlap or corruption
- 5TestWrite TX_BD_NUM = 0x80 (all TX): verify RXEN auto-disabled in MODER
- 6TestWrite TX_BD_NUM = 0x00 (all RX): verify TXEN auto-disabled in MODER

PASS CRITERIA

TX BD at 0x400 (first) and 0x58C (last) correctly stored and read

RX BD at 0x590 (first) and 0x7FC (last) correctly stored and read

RXEN auto-disables when TX_BD_NUM = 0x80

TXEN auto-disables when TX_BD_NUM = 0x00

Values > 0x80 are ignored (register retains previous value)

5- tc_miim_write_phy_reg

Programs a PHY configuration register by setting up MIADDRESS, MIITX_DATA, then asserting WCTRLDATA in MIICOMMAND. Verifies the full 64-bit MIIM frame appears on the MDC/MDIO interface with correct preamble, address, and data fields.

COMPONENTS USED

wb_slave_agent	mdio_agent	eth_reg_block	
(RAL)	eth_miim_write_seq	mdio_monitor	eth_scoreboard

TEST FLOW

- 1eth_miim_write_seqRAL write: MIIMODER.CLKDIV = 0x28 (MDC = CLK/40)
- 2eth_miim_write_seqRAL write: MIIADDRESS.FIAD = 0x01 (PHY addr), RGAD = 0x00 (control reg)
- 3eth_miim_write_seqRAL write: MIITX_DATA.CTRLDATA = 0x2100
- 4eth_miim_write_seqRAL write: MIICOMMAND.WCTRLDATA = 1

5wb_slave_driverIssues all four WISHBONE write cycles with ACK_O handshake(The driver waits for the DUT to generate ACK_O before starting the next write.)

6mdio_monitorCaptures MDC edges and samples MDIO bit stream

7mdio_monitorDecodes: 32-bit preamble(1s), ST=01, OP=01(write), FIAD, RGAD, TA=10, DATA

8eth_miim_write_seq poll MIISTATUS.BUSY via RAL read until cleared (reg_model.miistatus.read(status, value) performs:Physical bus read ,Receives value from DUT ,Updates mirror value automatically)

9mdio_agentSimulates PHY acknowledging write (drives MDIO during TA with value 2'b10 for write)

10eth_scoreboardVerifies decoded frame fields match programmed values

PASS CRITERIA

MDIO preamble = 32 ones (unless MIINOPRE set)

ST field = 2'b01

OP field = 2'b01 for write

FIAD[4:0] matches MIIADDRESS.FIAD written

RGAD[4:0] matches MIIADDRESS.RGAD written

TA = 2'b10 for write

DATA[15:0] matches MIITX_DATA.CTRLDATA

MIISTATUS.BUSY deasserts at end of operation

6- tc_miim_clkdiv_all

Iterates through all valid CLKDIV values (1–255) and for each value triggers a MIIM read operation, measures the MDC frequency, and verifies it equals $CLK/(2 \times CLKDIV)$.

COMPONENTS USED

wb_slave_agent mdio_agent	eth_reg_block (RAL)	eth_miim_clkdiv_seq
---------------------------	---------------------	---------------------

TEST FLOW

1eth_miim_clkdiv_seq For CLKDIV = 1 to 255: RAL write to MIIMODER.CLKDIV

2eth_miim_clkdiv_seq Trigger MIICOMMAND.RSTAT = 1 to start a read operation

3mdio_monitor Measures MDC period: records rising edge timestamps, computes frequency

4mdio_monitor Verifies $MDC_freq = SYS_CLK_freq / (2 \times CLKDIV) \pm 1$ cycle tolerance

5eth_miim_clkdiv_seqWait for BUSY = 0 before next iteration

PASS CRITERIA

MDC frequency matches formula for all 255 values

No MDC glitches (spurious edges) between operations

BUSY correctly asserted for full duration of each read

CLKDIV=0 treated as 1 or ignored (per implementation)

We need read operation as MDC does not run continuously. It only toggles when the MIIM module is actively performing an operation.

MDC is a divided version of CLK. The counter counts CLKDIV system clock cycles, then toggles MDC. One full MDC period = $2 \times CLKDIV$ system clock cycles.

7- tc_miim_scan_linkfail

Asserts SCANSTAT in MIICOMMAND to start continuous PHY status polling. The mdio_agent (PHY model) first returns link-OK, then drives LINKFAIL=1 in the status register. Verifies MIISTATUS.LINKFAIL reflects the PHY state and NVALID clears after first scan completes.

COMPONENTS USED

wb_slave_agent mdio_agent eth_reg_block (RAL) eth_miim_scan_seq

TEST FLOW

- 1eth_miim_scan_seqWrite MIICOMMAND.SCANSTAT = 1; MIIADDRESS.FIAD = PHY_ADDR
- 2mdio_agentFirst scan response: bit[2]=1 (link OK) in PHY status register
- 3eth_miim_scan_seqRAL read MIISTATUS: verify NVALID=1 (invalid until first scan done)
- 4mdio_monitorDetects end of first scan frame; NVALID should clear
- 5eth_miim_scan_seqRAL read MIISTATUS: verify NVALID=0, LINKFAIL=0
- 6mdio_agentSecond scan response: bit[2]=0 (link fail) in PHY status register
- 7eth_miim_scan_seqRAL read MIISTATUS: verify LINKFAIL=1
- 8eth_miim_scan_seqWrite MIICOMMAND = 0x00 to stop scan; verify BUSY deasserts

PASS CRITERIA

- NVALID=1 until first complete scan frame received
- NVALID=0 after first scan completes
- LINKFAIL tracks PHY bit[2] of status register
- BUSY stays asserted during entire scan operation
- BUSY deasserts only after scan explicitly stopped

8- tc_miim_stop_scan_after_read

Asserts **SCANSTAT in MIICOMMAND** to start continuous PHY status polling, then performs a sliding stop by clearing SCANSTAT immediately after scan begins. The mdio_agent (PHY model) responds to scan reads with changing link status (LINK OK → LINK FAIL). Verifies that scanning stops cleanly without initiating further MII transactions after SCANSTAT is cleared, for both MIINOPRE = 0 and MIINOPRE = 1 modes.

COMPONENTS USED

wb_slave_agent mdio_agent eth_reg_block (RAL) eth_miim_stop_scan_seq
mdio_monitor eth_scoreboard

TEST FLOW

- 1 eth_miim_stop_scan_seq RAL write: MIIMODER.CLKDIV = 0x28 (set MDC to measurable frequency), MIIMODER.MIINOPRE = 0 (preamble enabled for first pass)
- 2 eth_miim_stop_scan_seq RAL write: MIIADDRESS = 0x0101 (FIAD=1, RGAD=1 — PHY status register)
- 3 eth_miim_stop_scan_seq RAL write: MIICOMMAND = 0x00000001 (SCANSTAT=1 — start continuous scan)
- 4 mdio_monitor Detects first MDC rising edge — records scan start timestamp; confirms MDC frequency matches CLKDIV formula
- 5 mdio_agent Responds to scan frame: drives TA acknowledge (MDIO=0 on second TA bit), drives 16-bit PHY status = 0x0024 (link OK, bit[2]=1)
- 6 eth_miim_stop_scan_seq Waits for first MDC rising edge after SCANSTAT was written using @(posedge mdio_vif.MDC) — this marks MDC cycle 0 of frame 1

7	eth_miim_stop_scan_seq	Outer loop: preamble $\in$ {enabled, disabled} — for disabled preamble, repeat steps 1–6 with MIINOPRE=1 and frame_len=32
8	eth_miim_stop_scan_seq	Inner loop: stop_at_cycle from 0 to frame_len (0 to 64 with preamble, 0 to 32 without) — each iteration: (a) write MIICOMMAND=0x01 to restart scan, (b) wait exactly stop_at_cycle MDC rising edges using repeat(stop_at_cycle) @(posedge mdio_vif.MDC), (c) write MIICOMMAND=0x00 (clear SCANSTAT)
9	mdio_monitor	For each iteration: records timestamp of last MDC edge before stop; verifies no further MDC edges appear within 10 CLK cycles after stop command
10	eth_miim_stop_scan_seq	RAL read MIISTATUS in a polling loop: waits for BUSY=0 with timeout of 50,000 CLK cycles; if timeout fires, records the stop_at_cycle value that caused the hang
11	mdio_monitor	Verifies MDIO line returns to high-Z state within 2 MDC cycles of SCANSTAT being cleared; no spurious edges on MDIO after stop
12	eth_miim_stop_scan_seq	For stop positions where stop_at_cycle < frame_len: verifies exactly 0 complete scan frames were delivered to PHY (partial frame was in progress, aborted)
13	eth_miim_stop_scan_seq	For stop_at_cycle = frame_len (exactly at end): verifies exactly 1 complete scan frame was delivered; DUT completed current frame before stopping
14	eth_miim_stop_scan_seq	Recovery check: after BUSY=0, write MIICOMMAND.RSTAT=1 (single read), verify it completes normally — confirms MIIM state machine returned to idle cleanly
15	eth_scoreboard	After all iterations: verifies zero cases where BUSY failed to clear; zero cases where MDC continued after stop; zero cases where MDIO was not released to high-Z

PASS CRITERIA

BUSY deasserts within $128 \times \text{CLKDIV}$ CLK cycles after SCANSTAT cleared for every stop position

MDC stops toggling within 2 CLK cycles of SCANSTAT being cleared at every stop position

MDIO returns to high-Z within 2 MDC cycles after scan stops at every stop position

No MDC edges appear after stop for at least 100 CLK cycles (confirms MDC truly stopped, not just paused)

Recovery single-read operation (RSTAT) completes correctly after every stop position — confirms state machine returns to idle cleanly

With preamble disabled (MIINOPRE=1): all above hold with frame_len=32 instead of 64

Zero simulation timeouts across all $65 + 33 = 98$ iterations total (64+1 with preamble, 32+1 without)

9- tc_miim_stop_scan_after_write

Stop scan immediately after a write request is issued; sliding stop position across the write frame while both WCTRLDATA and SCANSTAT are simultaneously set.

COMPONENTS USED

wb_slave_agent	mdio_agent	eth_reg_block (RAL)	eth_miim_stop_scan_seq
mdio_monitor	eth_scoreboard		

TEST FLOW

- eth_miim_stop_scan_seq RAL write: MIIMODER.CLKDIV = 0x28, MIIMODER.MIINOPRE = 0 (preamble enabled for first pass)
- eth_miim_stop_scan_seq RAL write: MIIADDRESS = 0x0101 (FIAD=1, RGAD=1)
- eth_miim_stop_scan_seq RAL write: MIITX_DATA = 0x00002100 (PHY control register value: reset + loopback) — this is the data the write frame will carry

4	eth_miim_stop_scan_seq	RAL write: MIICOMMAND = 0x00000001 (SCANSTAT=1 only — start scan)
5	mdio_monitor	Confirms scan is running; detects MDC toggling, records scan_active = 1
6	mdio_agent	Responds to first scan frame with PHY status = 0x0024 (link OK) to confirm scan is operational before the test begins
7	eth_miim_stop_scan_seq	Waits for exactly one complete scan frame to confirm scan is stably running: repeat(frame_len) @(posedge mdio_vif.MDC)
8	eth_miim_stop_scan_seq	RAL write: MIICOMMAND = 0x00000005 (WCTRLDATA=1 AND SCANSTAT=1 simultaneously — both bits set at the same time) — this is the critical moment: a write command is injected while scan is running
9	mdio_monitor	Captures the MIICOMMAND write timestamp; begins monitoring MDIO to detect whether DUT starts a write frame or continues scanning
10	eth_miim_stop_scan_seq	Outer loop: preamble {enabled, disabled}
11	eth_miim_stop_scan_seq	Inner loop: stop_at_cycle from 0 to frame_len — for each iteration: (a) restore to step 8 state (scan running + WCTRLDATA pending), (b) wait stop_at_cycle MDC edges, (c) write MIICOMMAND = 0x00000004 (clear SCANSTAT, keep WCTRLDATA=1)
12	mdio_monitor	At each iteration: decodes the MDIO bit stream; determines whether a WRITE frame or a SCAN frame is currently in progress; records frame type and completion status
13	mdio_agent	For any write frame detected: drives TA field correctly (MDIO=high-Z then MDIO=0); does NOT drive data response (write frames have host driving the data, not PHY)
14	eth_miim_stop_scan_seq	After clearing SCANSTAT: polls MIISTATUS.BUSY until cleared with 50,000-cycle timeout
15	eth_scoreboard	Verifies write frame integrity: decodes ST=01, OP=01, FIAD=00001, RGAD=00001, TA=10, DATA=0x2100 from the captured MDIO bit stream; any bit mismatch flagged as write frame corruption
16	eth_scoreboard	Verifies exactly one complete write frame appears on MDIO regardless of when SCANSTAT was cleared; the write must not be silently dropped
17	eth_miim_stop_scan_seq	Verifies scan does not restart after write completes: monitors MDC for 200 CLK cycles after BUSY clears — no new MDC edges should appear since SCANSTAT was cleared
18	eth_miim_stop_scan_seq	Recovery check: write MIICOMMAND.RSTAT=1, verify single read completes normally — confirms state machine is in idle, not stuck in write or scan state

PASS CRITERIA

Write frame appears on MDIO and completes correctly for every value of stop_at_cycle (0 to 64 with preamble, 0 to 32 without) — write is never silently dropped regardless of when SCANSTAT is cleared

Write frame content is bit-perfect: ST=01, OP=01 (write), FIAD matches MIIADDRESS.FIAD, RGAD matches MIIADDRESS.RGAD, TA=10, DATA=0x2100 for all sliding positions

Scan does not restart after write frame completes (SCANSTAT was cleared)

BUSY deasserts after write frame completes, not prematurely after scan stops

No overlapping scan and write frames on MDIO (no frame interleaving or corruption)

With preamble disabled: write frame = 32 MDC cycles; all above still hold

Recovery read operation completes correctly after every stop position

10-tc_miim_stop_scan_after_scan

Stop scan at the exact boundary between the 1st and 2nd completed scan frames, and between the 2nd and 3rd, using a narrow sliding window centered on each frame boundary.

COMPONENTS USED

wb_slave_agent	mdio_agent	eth_reg_block (RAL)	eth_miim_stop_scan_seq
mdio_monitor	eth_scoreboard		

Test flow

- 1 eth_miim_stop_scan_seq RAL write: MIIMODER.CLKDIV = 0x28, MIIMODER.MIINOPRE = 0
- 2 eth_miim_stop_scan_seq RAL write: MIIADDRESS = 0x0101 (FIAD=1, RGAD=1 — PHY status register)
- 3 eth_miim_stop_scan_seq RAL write: MIICOMMAND = 0x00000001 (SCANSTAT=1 — start continuous scan)
- 4 mdio_monitor Detects first MDC rising edge; begins counting MDC edges from this point as the absolute frame counter reference total_mdc_count
- 5 mdio_agent For every scan frame that arrives: drives TA acknowledge then drives PHY status = 0x0024 (link OK, LINKFAIL=0) for all frames; this ensures scan runs cleanly during the sliding window measurement
- 6 eth_miim_stop_scan_seq **Outer loop: preamble ∈ {enabled, disabled}** — frame_len = 64 with preamble, frame_len = 32 without
- 7 eth_miim_stop_scan_seq **Sub-test A — boundary between frame 1 and frame 2:** inner loop with stop_offset from -5 to +5 MDC cycles relative to frame_len (absolute stop cycle = frame_len + stop_offset, range 59–69 with preamble, 27–37 without)
For each stop_offset iteration of Sub-test A: (a) write MIICOMMAND=0x01 to restart fresh scan, (b) @(posedge mdio_vif.MDC) to sync to first edge, (c) repeat(frame_len + stop_offset) @(posedge mdio_vif.MDC) to reach stop position, (d) write MIICOMMAND=0x00 to clear SCANSTAT
- 8 eth_miim_stop_scan_seq
- 9 mdio_monitor For each Sub-test A iteration: counts total complete scan frames that appeared on MDIO before MDC stopped; records n_complete_frames
- 10 eth_miim_stop_scan_seq For Sub-test A with stop_offset ≤ 0 (stop at or before boundary): verifies n_complete_frames == 1; DUT stopped after frame 1, frame 2 never started
- 11 eth_miim_stop_scan_seq For Sub-test A with stop_offset > 0 (stop after boundary): verifies n_complete_frames == 2; DUT committed to frame 2 before seeing the stop, must complete it
- 12 eth_miim_stop_scan_seq For all Sub-test A iterations: polls MIISTATUS.BUSY until cleared; verifies BUSY clears correctly regardless of stop_offset; no timeout
- 13 mdio_monitor For all Sub-test A iterations: verifies no partial frames appear — every frame that starts must complete fully; if frame 2 started it must be 64 complete MDC cycles (not 60 or 70)
- 14 eth_miim_stop_scan_seq **Sub-test B — boundary between frame 2 and frame 3:** inner loop with stop_offset from -5 to +5, absolute stop cycle = (2 × frame_len) + stop_offset (range 123–133 with preamble, 59–69 without)
- 15 eth_miim_stop_scan_seq For each Sub-test B iteration: same procedure as steps 8–13 but waiting 2×frame_len + stop_offset MDC edges before clearing SCANSTAT
- 16 mdio_monitor For Sub-test B with stop_offset ≤ 0: verifies n_complete_frames == 2; for stop_offset > 0: verifies n_complete_frames == 3
- 17 eth_miim_stop_scan_seq After Sub-test B completes: verifies NVALID=0 in MIISTATUS (at least one full scan completed so data is valid) and LINKFAIL=0 (PHY returned link OK on all frames during test)
- 18 eth_miim_stop_scan_seq Recovery check for both sub-tests: after each BUSY=0, write MIICOMMAND.RSTAT=1, verify single read completes; confirms state machine returned to idle and did not get stuck at frame boundary
- 19 eth_scoreboard Final check: total frames observed = sum of all n_complete_frames across both sub-tests and both preamble modes; verifies count matches expected value of (1+2)×11 + (2+3)×11 = 33+55 = 88 complete frames across all iterations

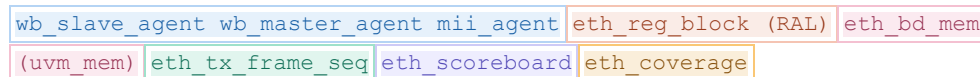
Pass criteria

- Sub-test A with stop_offset ≤ 0: exactly 1 complete scan frame transmitted in all 6 cases (offsets -5,-4,-3,-2,-1,0)
- Sub-test A with stop_offset > 0: exactly 2 complete scan frames transmitted in all 5 cases (offsets +1,+2,+3,+4,+5)
- Sub-test B with stop_offset ≤ 0: exactly 2 complete scan frames transmitted in all 6 cases
- Sub-test B with stop_offset > 0: exactly 3 complete scan frames transmitted in all 5 cases
- No partial frames on MDIO at any stop position — every frame that starts must be exactly frame_len MDC cycles long
- BUSY deasserts after last complete frame finishes for all 44 iterations across both sub-tests
- NVALID=0 after test (at least 1 complete scan confirmed valid data)
- LINKFAIL=0 throughout (mdio_agent drove link-OK on every frame)
- Recovery single read completes correctly after every iteration
- With preamble disabled (frame_len=32): all frame count checks hold with boundary at cycle 32 and 64 instead of 64 and 128
- Zero simulation timeouts across all 44 iterations × 2 preamble modes = 88 total iterations

11- tc_tx_basic_frame

Transmits a single 64-byte Ethernet frame. The wb_master_agent holds the frame bytes in its SRAM (associative array). A TX BD is programmed pointing to that SRAM address. After TXEN=1, the DUT reads the frame via WISHBONE master and transmits nibbles on MTxD. The mii_agent captures and reassembles the full frame.

COMPONENTS USED



TEST FLOW

- 1eth_tx_frame_seqWrite 64 bytes of frame data into wb_master_agent SRAM at address 0x0000_1000 (backdoor write to SRAM model)
- 2eth_tx_frame_seqRAL write MODER: TXEN=0, CRCEN=1, PAD=1, FULLD=0 (configure before enable)
- 3eth_tx_frame_seqProgram TX BD at 0x400: LEN=64, CRC=1, PAD=1, IRQ=1, WR=1, RD=0
- 4eth_tx_frame_seqWrite TXPNT at 0x404 = 0x0000_1000
- 5eth_tx_frame_seqSet TX BD RD=1 (ready bit) — this arms the transmitter
- 6eth_tx_frame_seqRAL write MODER.TXEN = 1
- 7DUTSees RD=1, issues WISHBONE master reads: M_ADDR_O=0x1000, M_WE_O=0
- 8wb_master_agentResponds to each read burst with frame bytes from SRAM, asserts M_ACK_I
- 9DUTFills TX FIFO (16 words), begins transmitting: MTxEn=1, MTxD = preamble nibbles
- 10mii_monitorCaptures MTxD stream: 14 nibbles preamble (0x5), 2 nibbles SFD (0xD), then data nibbles
- 11DUTTransmits data nibbles (LSB first), then 4-byte CRC nibbles; deasserts MTxEn
- 12DUTWrites TX BD: RD=0, RTRY=0, DF=0, CS=0; asserts INTA_O (IRQ=1)
- 13eth_scoreboardReconstructs frame from captured MII nibbles; compares against original SRAM bytes
- 14eth_scoreboardRecalculates expected CRC-32; verifies appended CRC matches

PASS CRITERIA

Preamble = 7 bytes of 0x55, SFD = 0xD5
Frame payload matches exactly what was written to SRAM
CRC-32 appended and mathematically correct
MTxEn deasserts on same cycle as last CRC nibble

TX BD RD bit cleared to 0 after transmission
INTA_O asserts and INT_SOURCE.TXB=1
M_WE_O=0 for all DMA reads (read-only on TX path)

12-tc_tx_short_frame_pad on and off

Transmits a 20-byte payload (below 64-byte MINFL). Runs twice: once with PAD=1 (should pad to 64 bytes then append CRC), once with PAD=0 (transmits as-is with or without CRC based on CRCEN).

COMPONENTS USED

wb_slave_agent	wb_master_agent	mii_agent	eth_reg_block
(RAL)	eth_tx_frame_seq	eth_scoreboard	

TEST FLOW

```
1eth_tx_frame_seqWrite 20-byte payload to SRAM at 0x0000_2000
2eth_tx_frame_seqRun A: MODER.PAD=1, CRCEN=1; program TX BD LEN=20, RD=1
3DUTTransmits 20 data bytes, then zero-pads to 60 bytes (MINFL=64 minus 4-byte CRC), then appends 4-
byte CRC
4mii_monitorCaptures total frame length = 64 bytes (preamble/SFD excluded)
5eth_tx_frame_seqRun B: MODER.PAD=0, CRCEN=1; same TX BD
6DUTTransmits 20 data bytes, then 4-byte CRC (no padding) — total payload 24 bytes
7eth_scoreboardVerifies frame lengths match expected for each run
```

PASS CRITERIA

Run A: transmitted frame = 64 bytes total (20 data + 40 zeros pad + 4 CRC)
Run A: CRC computed over padded frame (64 bytes)
Run B: transmitted frame = 24 bytes (20 data + 4 CRC, no pad)
TX BD status bits correct (no error flags) in both runs

13- tc_tx_underrun

Forces the wb_master_agent to insert random WAIT states (withhold M_ACK_I) so the TX FIFO drains empty while transmission is in progress. Verifies the DUT aborts transmission and sets the UR bit in the TX BD.

COMPONENTS USED

wb_slave_agent	wb_master_agent	mii_agent	eth_reg_block
(RAL)	eth_tx_underrun_seq	eth_scoreboard	

TEST FLOW

```
1eth_tx_underrun_seqProgram 512-byte frame in SRAM; set TX BD RD=1, IRQ=1
2eth_tx_underrun_seqConfigure wb_master_agent to hold M_ACK_I low for 500+ cycles after 4th read
3DUTStarts TX, reads first 4 words into FIFO, begins transmitting
4DUTFIFO empties while waiting for M_ACK_I; TxAbort asserted internally
5mii_monitorRecords MTxEn deasserting mid-frame (incomplete transmission)
6DUTWrites TX BD: RD=0, UR=1; asserts INTA_O
7eth_tx_underrun_seqRAL read INT_SOURCE: verify TXE=1
8eth_tx_underrun_seqRAL read TX BD status word: verify UR bit=1
```

PASS CRITERIA

TX BD UR bit set to 1

INT_SOURCE.TXE = 1 (transmit error interrupt)

MTxEn deasserts cleanly (no partial nibble)

TX BD RD cleared to 0

No subsequent frame corruption — next TX BD works correctly after recovery

14- tc_tx_collision_retry

With MODER.FULLD=0, the mii_agent asserts MColl asynchronously during frame transmission. Verifies the DUT sends a jam pattern, performs binary exponential backoff, and retries. Repeats up to MAXRET times.

COMPONENTS USED

wb_slave_agent	wb_master_agent	mii_agent	eth_reg_block
(RAL)	eth_collision_seq	eth_scoreboard	

TEST FLOW

- 1eth_collision_seqMODER.FULLD=0, COLCONF.MAXRET=15, COLCONF.COLLVALID=0x3F
- 2eth_collision_seqProgram TX BD with 128-byte frame, RD=1
- 3eth_collision_seqEnable TXEN, wait until MTxEn=1 and 10 nibbles transmitted
- 4mii_agentAssert MColl asynchronously (simulates collision on wire)
- 5mii_monitorCaptures jam pattern: 4 bytes 0x99999999 on MTxD
- 6DUTStops TX, waits random backoff (eth_random.v: binary exponential)
- 7mii_monitorMeasures backoff time = random × slot_time
- 8DUTRetransmits from beginning of frame (preamble again)
- 9eth_collision_seqLet retry succeed (no MColl); verify TxDone and RTRY count in TX BD

PASS CRITERIA

Jam pattern = 0x99999999 (4 bytes) immediately after collision

Backoff time is random within $[0, 2^N - 1]$ slot times for Nth collision

RTRY field in TX BD = number of collisions before success

TX BD RL=1 and abort if MAXRET+1 collisions injected

WillTransmit signal deasserts during backoff (RX can proceed)

